

Warum - Weshalb - Wie

1. Kapitel - Introduction and Goals

Unternehmen heißt youRide

Fuer das MVP (Minimal viable product) gehoeren Registrierung, Fahrtsuche, Fahrt buchen, automatische Fahrerzuordnung, Anfrage annehmen/ablehnen, Tracking, Fahrt abbrechen, Fahrt abschließen, Historie, Payment, Driver Verification, Admin-Funktionen und Reporting.

Top 3 Quality Goals: Functional Suitability, Usability, Scalability

Stakeholder: Wir 3 Unternehmer/Developer, Fahrer, Kunden, Devops/Network Mitarbeiter, Controlling Mitarbeiter

Business Ziel: Schnell zahlende Kunden gewinnen, MVP bauen, Kosten sparen und spaeter durch Wachstum Investoren ueberzeugen

Kapitel 2 - Architecture Constraints

Wo laeuft youRide?: auf einem gemieteten Linux-Server. Fuer die Skalierung werden wir dann auf Cloud Services setzen und den Server fuer das Backup behalten

TechStack: Keine feste externe Vorgabe, aber unsere Entscheidung ist Java Backend, Angular Frontend, MySQL und REST API

Mitarbeiter: Wir 3 Unternehmer auch Developer, einen Devops/Network Mitarbeiter und einen Mitarbeiter fuer das Rechnungswesen/Controlling

Rechtlicher Rahmen: GDPR (General Data Protection Regulation) von der EU fuer Data Governance

Kosten: Aus Startup-Gruenden muessen laufende Kosten so gering wie moeglich gehalten werden

Kapitel 3 - System Scope and Context

Systemgrenze: youRide umfasst Mobile App/Frontend, Backend-Monolith, MySQL-Datenbank und Admin-Funktionen.

Business Context: Kunden, Fahrer, Admins, Controlling, externer Auth Provider und Stripe.

Technical Context: HTTPS/REST für normale App-Kommunikation, WebSocket over TLS für Live-Tracking, HTTPS zu Auth/Stripe, interne DB-Verbindung zu MySQL.

Kapitel 4 - Solution Strategy

Architektur Stil: Modular Monolith, also Monolith nach aussen, aber intern klare Module

Technologie Entscheidungen: Java Backend, Angular Frontend, MySQL, REST, Websocket, Stripe, externer Auth Provider

Skalierungsstrategie: erst gemieteter Linux-Server, spaeter Cloud Services, Server bleibt Backup

Massnahmen fuer Reliability/Security: Unit Tests, Integration Tests, Penetration Tests sowie GDPR orientierte Data Governance

Kapitel 5 - Building Block View

Level 1: youRide Mobile App, youRide Backend Monolith, MySQL Database, External Auth Provider, Stripe

Level 2: Backend Module: Identity/Auth Integration, Customer Management, Driver Management & Verification, Ride Management & Matching, Live Tracking, Payment Integration, Administration & Reporting, Persistence

Level 3: Ride Management & Matching als White Box: Ride Application Service, Ride Aggregate, Matching Service, Price Calculation, Ride Status Policy, Ride Repository Port, Payment Handoff

Kapitel 6 - Runtime View

1: Kunde bucht fahrt mit automatischer driver matching

2: Fahrer akzeptiert fahrt und live tracking beginnt

3: Fahrt ist beendet und Zahlprozess startet ueber Stripe

Kapitel 7 - Deployment view

Wo laeuft alles?: auf dem gemieteten Linux-Server: also Backend, MySQL, Webserver/Reverse Proxy

Wo laeuft die App?: Als Mobile App auf den Geraeten der Kunden und Fahrer

Admin/Controlling: Browser-basiertes Frontend fuer interne Admin- und Reporting-Aufgaben

Reverse Proxy: Wir verwenden Nginx (open source und sehr schnell)

Backups: MySQL-Backup und Konfigurations-Backup auf dem Server plus externe verschluesselte Off-Server-Kopie

Entwicklungsumgebungen: Dev lokal, Test/Staging auf Server, Production auf Server

Kapitel 8 - Cross Cutting Concepts

- 1: Modular monolith boundaries
- 2: Security and authentication
- 3: GDPR-oriented Data Governance
- 4: Error handling and Logging

Kapitel 9 - Architecture Decisions

- 1: Modular Monolith statt Microservices
- 2: Gemieteter Linux Server als erstes, spaeter cloud services
- 3: Externe Anbieter fuer authentication statt self-build
- 4: Stripe fuer Payment statt selbst gebautem Payment

Kapitel 10 - Quality Requirements

Performance: Antwortzeit unter 2 Sekunden von search/booking

Live Tracking: GPS updates zwischen 2-3 Sekunden

Usability: Fahrt soll ohne Anleitung buchbar sein, also so intuitiv wie moeglich

Scalability: Das System soll mit wachsendem Kundenstamm, mehr Fahrern und mehr Fahrten umgehen koennen. Gleichzeitig ist die monolithische Architektur hier ein Risiko, weil einzelne Funktionen nicht unabhaengig voneinander skaliert werden koennen

Security: Geschuetzte REST/WebSocket Endpunkte, externer Auth Provider, rollenbasierte Zugriffe und Data Governance mit GDPR

Kapitel 11 - Risks and Technical Debts

- 1: Monolithische Architektur kann mit der Zeit zu komplex werden (Aenderungen, Tests,...)
- 2: Ein gemieteter Server als Bottleneck oder Single Point of Failure
- 3: Abhaengigkeiten von externen Anbietern (Authentication und Stripe)
- 4: GDPR/Privacy Risiko, weil Kundenprofile, Fahrerprofile, Standortdaten, Fahrthistorie und Payment-Referenzen sensible Daten sind

- 5:** Skalierung mit monolithische Architektur aufwendiger/schwer realisierbar
- 6:** Backup/Restore Risiko, wenn Backups nicht vollstaendig, verschluesselt, extern gespeichert und regelmaessig getestet werden